

```

{
    public delegate void EventDelegate(SpecializedEvent
e);
    EventDelegate backingEvent;
    public event EventDelegate MyEvent
    {
        add { backingEvent += value; backingEvent +=
value; }
        remove { backingEvent -= value; }
    }
    protected void OnMyEvent()
    {
        backingEvent(this);
    }
    // Normally we would not expose a way to trigger
the event to clients;
    // this is purely for testing purposes.
    public void TriggerEvent() { OnMyEvent(); }
}
public class SpecializedEventTest
{
    public static void EventDelTest(SpecializedEvent e)
    {
        Console.WriteLine("In Test");
    }
    public static void Test()
    {
        SpecializedEvent e = new SpecializedEvent();
        e.MyEvent += EventDelTest;
        e.TriggerEvent();
    }
}

```

Note that we do not invoke the user-defined event directly when we want to trigger the event like with default events. We invoke the backing delegate instead. Both kinds of events do not expose a way to invoke the event outside of its containing class, even to inheriting subclasses. An idiom to provide this behavior is to declare a protected method OnX that triggers the X event, e.g., OnMyEvent above.